

0.12. Une lettre toutes les secondes

Écrire **DANS** la boucle de jeu, et compter les images pour mesurer le temps.

1. Ce qu'il faut comprendre

Jusqu'ici, on écrivait **avant** la boucle de jeu, une fois pour toutes. Ici, on écrit **dedans** : à chaque image, la case (0, 0) reçoit la lettre du moment.

La console ne connaît ni les secondes ni les millisecondes : elle compte les images. Il y en a **60 par seconde** ; une image dure donc $1000 \div 60 \approx 16,7$ millisecondes. `image()` attend l'image suivante : chaque tour de boucle dure une image.

`IMAGES_PAR_SECONDE` et `SECONDES` donnent un nom à ces nombres. `SECONDES * IMAGES_PAR_SECONDE` vaut $1 \times 60 = 60$: quand `images` atteint 60, une seconde est passée. Pour changer la vitesse, on ne touche qu'à `SECONDES`.

Limite : `images` est un `uint8_t`, qui ne dépasse pas 255. On compte donc au plus 255 images, environ **4 secondes** : `SECONDES = 5` donnerait 300, trop grand. Le chapitre suivant montre `attendre()`, qui n'a pas cette limite.

Chaque nouvelle lettre **écrase** la précédente, comme au 0.3. Après Z, `lettre` revient à 0 : on repart de A.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1364 octets** de cartouche.

1. UNE CONSTANTE — IMAGES_PAR_SECONDE

```
const uint8_t IMAGES_PAR_SECONDE = 60; // la console affiche 60 images par seconde
```

2. UNE CONSTANTE — SECONDES

```
const uint8_t SECONDES = 1; // le temps entre deux lettres, en secondes
```

3. UNE VARIABLE — LETTRE

```
uint8_t lettre = 0; // l'indice de la lettre affichée
```

4. UNE VARIABLE — IMAGES

```
uint8_t images = 0; // les images comptées depuis la dernière lettre
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

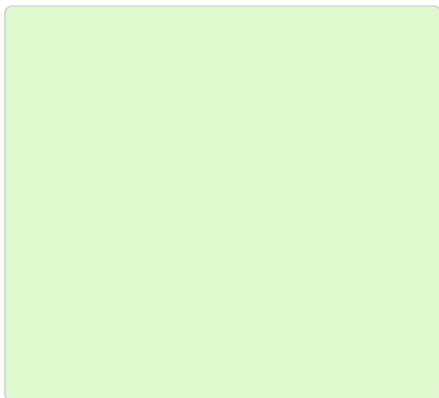
```
int main() {
    while (true) {
        image();          // attend l'image suivante : environ 16,7 ms
        images++;         // une image de plus

        if (images == SECONDES * IMAGES_PAR_SECONDE) { // 1 × 60 = 60 images = 1 seconde
            images = 0;          // on recommence à compter
            lettre++;            // la lettre suivante
            if (lettre == sizeof(ALPHABET)) lettre = 0; // après Z, on revient à A
        }

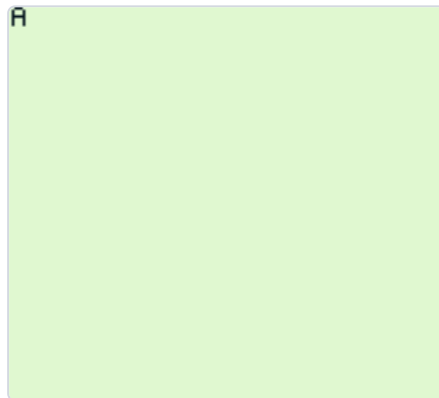
        poser(0, 0, ALPHABET[lettre]); // la même case : la lettre écrase la précédente
    }
}
```

3. Ce qui se passe, étape par étape

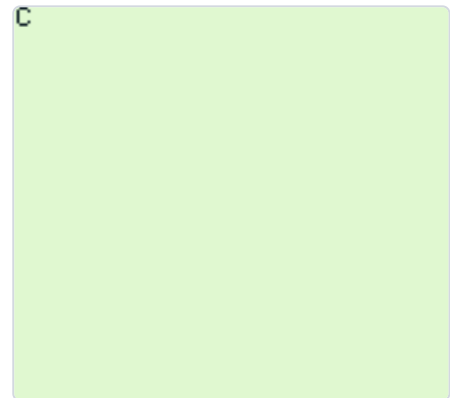
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

En haut à gauche : A, puis B une seconde plus tard, puis C...

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ A au départ
 - ✓ B une seconde plus tard
 - ✓ C la seconde d'après
-

Ensuite : **0.12.1. Une lettre toutes les secondes — de base, ailleurs** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++